

Pre-Lecture Assignment 4 — Value-Based RL

535510 Reinforcement Learning | NYCU CS

1) Q-learning: Derivation and Interpretations

The Q-learning update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(s_t, a_t) [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

Interpretation 1 — Approximate Q-Value Iteration (Q-VI). In the model-known setting, Q-VI iterates $Q(s,a) \leftarrow R + \gamma \sum P(s'|s,a) \max_{a'} Q(s',a')$. Since P and R are unknown, we approximate: the immediate reward R by the sampled r_t , and the expectation over transitions by the single observed s_{t+1} . A step-size α then interpolates between the old estimate and the new sample target, yielding the Q-learning update exactly.

Interpretation 2 — Stochastic correction of the Bellman (TD) error. Rewrite the update as $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \delta_t$, where $\delta_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)$ is the *temporal-difference error* — how far the current Q deviates from the Bellman optimality equation. Q-learning performs one stochastic gradient step to reduce this error, analogous to stochastic gradient descent on the squared Bellman error.

Why is Q-learning off-policy? The *target policy* being learned is always greedy: $\pi(s) = \arg\max_a Q(s,a)$ — reflected by the max operator in the update — while the *behaviour policy* collecting data can be any exploratory policy (e.g. ϵ -greedy). Because the policy being evaluated differs from the policy generating experience, Q-learning is *off-policy*.

2) DQN Loss Function: Components and Design Principle

$$L(w) := \mathbb{E}_{(s, a, r, s') \sim p} \left[\frac{1}{2} \left(r + \gamma \max_{a' \in \mathcal{A}} \hat{Q}(s', a'; \hat{w}) - \hat{Q}(s, a; w) \right)^2 \right]$$

Component meanings:

- $\hat{Q}(s,a; w)$ — the *online Q-network* with trainable weights w ; predicts the current action-value for (s, a) .
- $r + \gamma \max_{a'} \hat{Q}(s', a'; \hat{w})$ — the *TD target*, a one-step Bellman backup used as the regression label for the online network.
- $\hat{w}$ (**target network**) — a periodically-copied, frozen snapshot of w . Holding $\hat{w}$ fixed between updates prevents the regression target from moving every step, which would destabilise training.
- $(s,a,r,s') \sim p$ (**experience replay**) — transitions sampled i.i.d. from a replay buffer p , breaking temporal correlations and allowing data reuse.
- $\frac{1}{2}(\cdot)^2$ — squared TD error; minimising it drives $\hat{Q}(s,a;w)$ toward the TD target.

Design principle. The loss treats Q-learning as *bootstrapped regression*: the TD target acts as a (temporarily fixed) label and the online network is fitted to it. Two stabilisation mechanisms address the divergence problem of vanilla Q-learning with function approximation: (1) **experience replay** decorrelates samples and reuses data efficiently; (2) the **target network** provides stable regression targets, preventing the feedback loop that causes divergence.

3) Noisy Net (Rainbow Paper)

Standard DQN explores with ϵ -greedy, which applies uniform random perturbations and is difficult to tune, especially in sparse-reward settings (e.g. Montezuma's Revenge). **Noisy Nets** (Fortunato et al., 2017) replace each linear layer $y = b + Wx$ with a *noisy linear layer*:

$$y = (b + Wx) + (b_{\text{noisy}} \odot \epsilon^b + (W_{\text{noisy}} \odot \epsilon^w) x)$$

where ϵ^b and ϵ^w are random noise variables sampled per forward pass, and $\odot$ is the element-wise product. The noise magnitude parameters $(W_{\text{noisy}}, b_{\text{noisy}})$ are *learnable*, so the network adaptively reduces noise where it is already confident — a form of **self-annealing**. This enables **state-conditional, parametric exploration** without a manually scheduled ϵ . In Rainbow, all linear layers are replaced by noisy layers and ϵ is set to 0, with factorised Gaussian noise used to reduce the number of independent noise variables.